LAB ACTIVITY · TEAM ORIENTEERING PROBLEM

Savings, GRASP and local search for the team orienteering problem

The heuristic of Panadero et al. (2021), rebuilt in Rust and run on all 387 instances of Chao et al. (1996)

Jesús Inurria Porrero · Javier Salvatierra Corchado

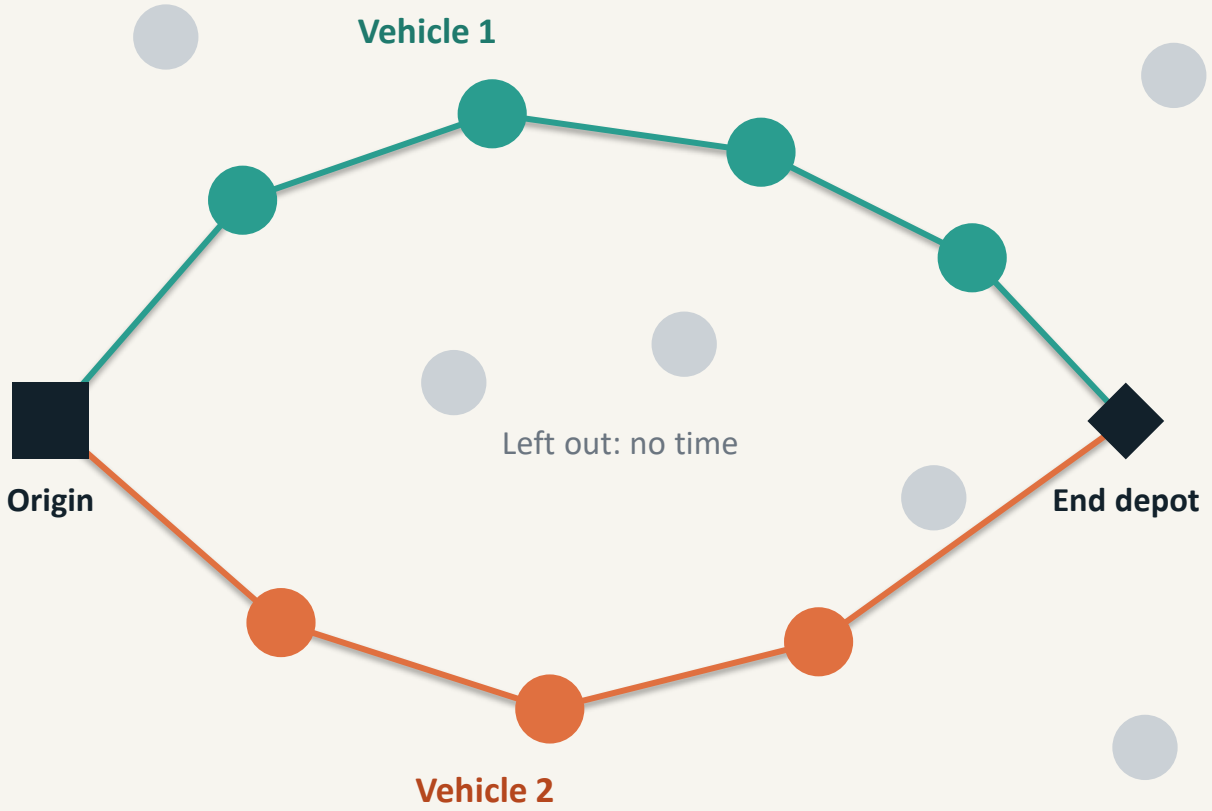
The team orienteering problem

Every customer carries a reward.

m vehicles leave the origin and must reach the end depot.

Each route must fit in a time limit tmax, so some customers stay unvisited.

Maximise reward, not coverage.



Benchmark: 387 instances of Chao et al. (1996) in 7 sets, 21 to 102 nodes, 2 to 4 vehicles. In 34 of them no customer fits even alone, so 353 count.

The PJS savings heuristic

ENRICHED SAVING OF ARC (i, j)

$$s'_{ij} = \alpha \cdot s_{ij} + (1 - \alpha) \cdot (u_i + u_j)$$

s_{ij} = time saved by joining two routes

$u_i + u_j$ = reward the joined route collects

1 Dummy solution

one route origin → i → end per customer

→

2 Sort the arcs

by s'_{ij}, highest first, both directions

→

3 Merge greedily

join two routes if the result fits in tmax

→

4 Keep m routes

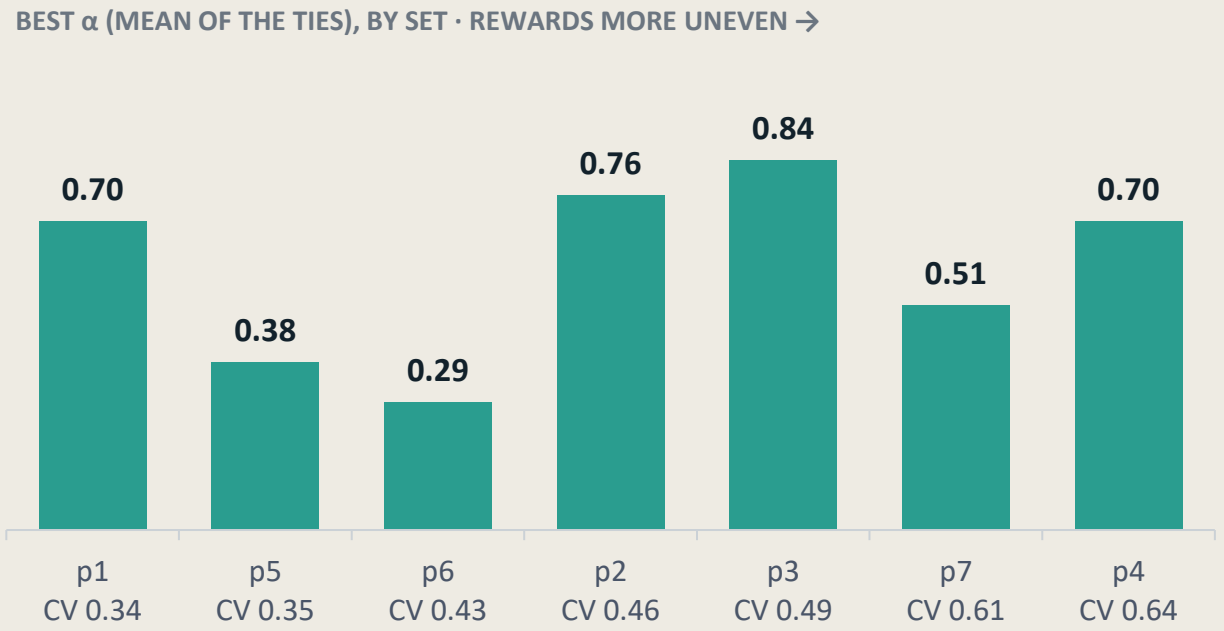
the m with the most reward

α trades distance against reward. The paper gives no value for it.

Panadero et al. (2021), section 4. With origin ≠ end depot, $s_{ij} \neq s_{ji}$, so both directions are kept, as in Li et al. (2022).

The paper's rule for α does not hold here

Panadero: *"α close to zero with heterogeneous rewards and close to one with homogeneous rewards."*



+0.19

correlation between reward CV and best α over 285 instances: weak, and with the wrong sign

68 %

of the best reward is all α = 0 reaches in p4, the most uneven set

So we run all 21 values α = 0, 0.05, …, 1 and keep the best per instance.

Deterministic PJS. The 68 instances where all α tie are left out. Rewards are fixed per set, so there are only 7 distinct CVs.

GRASP: the same heuristic, slightly random

Instead of always taking the arc with the highest saving, pick one at random, biased towards the top of the list. Repeat the construction many times and keep the best solution.

Geometric (Panadero)

$k \sim \text{Geom}(\beta),\ \beta = 0.3$

Take the k-th remaining arc. Rank matters, the size of the gap between savings does not.

Softmax (our proposal)

$P(\text{arc}) \propto \exp(s' / \tau),\ \tau = 1$

Each arc in proportion to its saving. Close savings get close chances.

BUDGETS TESTED, PER RUN (ONE RUN = ONE THREAD OF PANADERO'S ALGORITHM 1)

10 iterations

main grid: 21 α × 20 runs

100 iterations

same seeds, 10× longer

1 second

like the paper: ~40,000–56,000 iterations

Every run starts from the deterministic solution and is seeded once, so results are reproducible (except the 1 s runs).

Local search, applied to the elite at the end

Which solutions get it

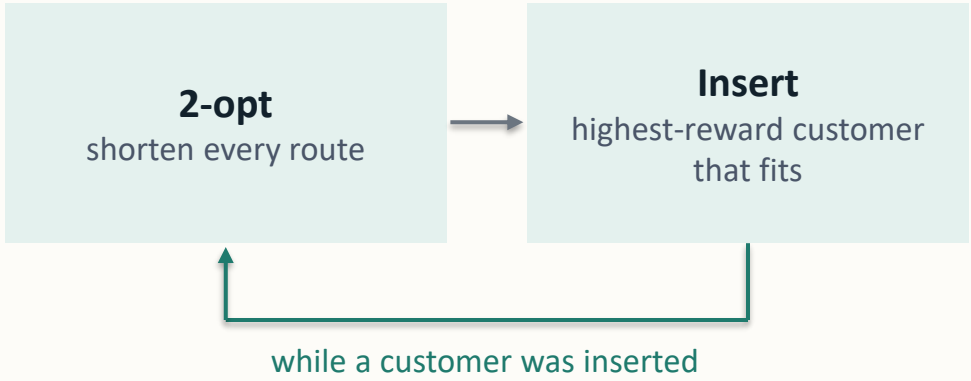
The deterministic solutions of the 3 best α.

For each α and randomisation, the best solution of the 3 best runs.

Ties at third place all enter. Duplicates (same set of customers) are dropped.

About 53 elite per instance; all 18,757 take 0.4 s in total.

What it does, in a loop



The time 2-opt frees is spent on the next insertion. Neither move can lose reward, so there is no acceptance rule.

Every elite is kept as a starting point: the best one before local search is often not the best after it.

Insertion: cheapest position in any route; ties in reward go to the smallest added time.

The whole method, per instance

PER INSTANCE · EACH RUN AND EACH ELITE IS A PARALLEL TASK

```
for α in 0, 0.05, …, 1:                 // 21 values
    det[α] ← PJS(α)                     // greedy
    for rule in {geometric, softmax}:
        for r in 1 … 10:                // 1 run = 1 task
            rng ← seed(α, rule, r)      // once per run
            best ← det[α]
            repeat 10 times:            // in series
                S ← PJS(α, rule, rng)
                if u(S) > u(best): best ← S
            run[α, rule, r] ← best

elite ← det of the 3 best α
      ∪ 3 best runs per (α, rule)
      − duplicates                      // same customers
for e in elite: e ← LOCAL_SEARCH(e)     // parallel
return the elite with the most reward
```

LOCAL_SEARCH(S)

```
repeat:
  for each route R of S:
    2-OPT(R)
    // reverse a segment
    // while R gets shorter
  k ← unvisited customer
      with most reward
      that fits in a route
      // tie: least time added
  if no k: return S
  insert k at its
      cheapest position
```

u(S) = total reward. rayon spreads the tasks (21 per instance and α) over 12 threads; the iterations of a run go in series.

Why Rust and not Python

Compute speed

Compiled to native code, no interpreter in the inner loop. We did not time a Python version, but the main grid, about 1.6 million constructions plus local search, runs in 140 s.

Parallelism with rayon

Going parallel is a one-word change, and the compiler rejects data races:

```
files.iter().map(load)       // 1 thread
files.par_iter().map(load)   // all 12
```

The code is written mostly by AI

Python's big advantage is that people write it faster. When an AI writes most of the code, that advantage mostly disappears, and the faster language costs nothing extra.

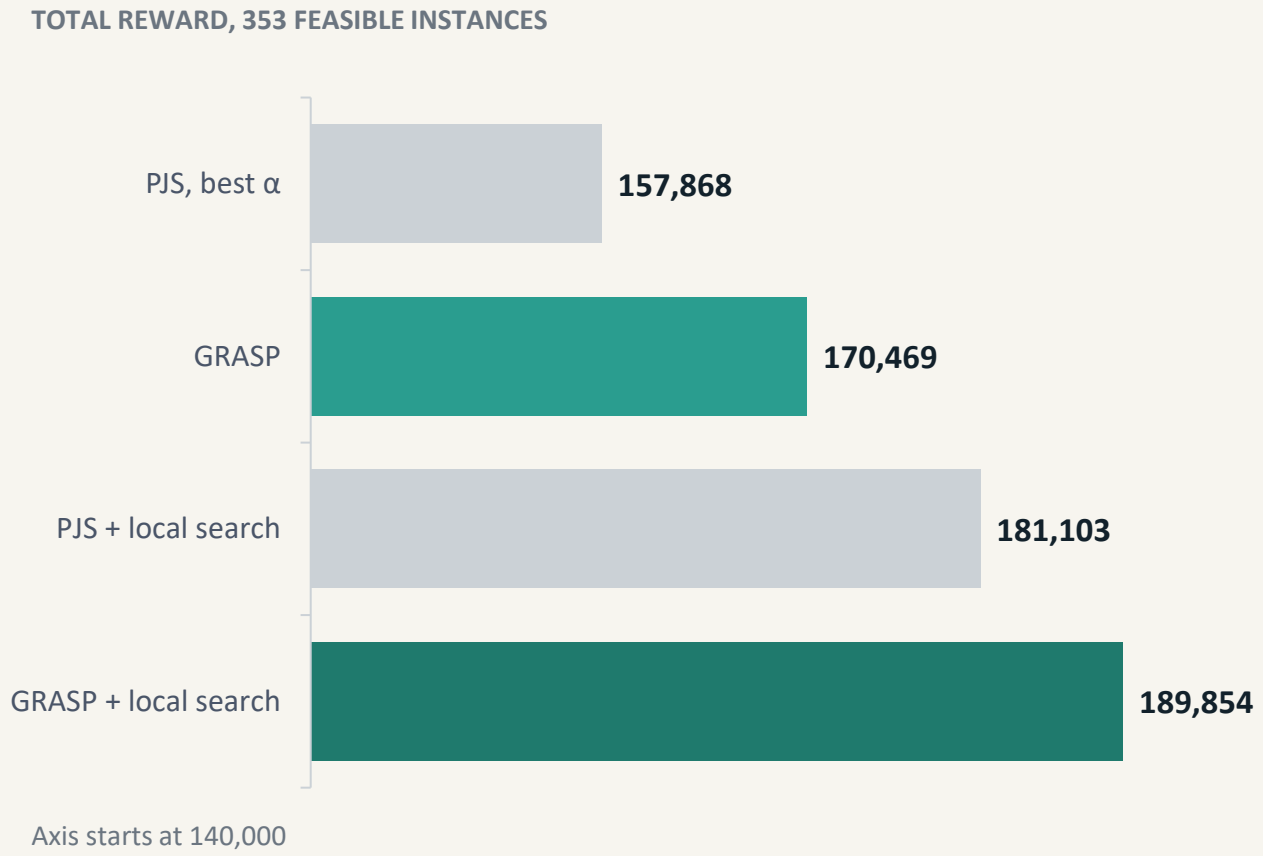
The compiler guides the AI

rustc names the exact line, explains the error and often suggests the fix. Type and ownership errors show up at compile time, not halfway through a two-minute run.

Fast enough to run every α, every run and every elite on all 387 instances in about two minutes.

Release build: fat LTO, one codegen unit, target-cpu=native. Python was used once, as an independent check: 0 differences in 38,700 runs.

Each layer adds reward: +20 % over plain PJS



+8.0 %

GRASP alone over the best deterministic α

93.6 %

of the elite improve with local search, +18 % on average

163 / 353

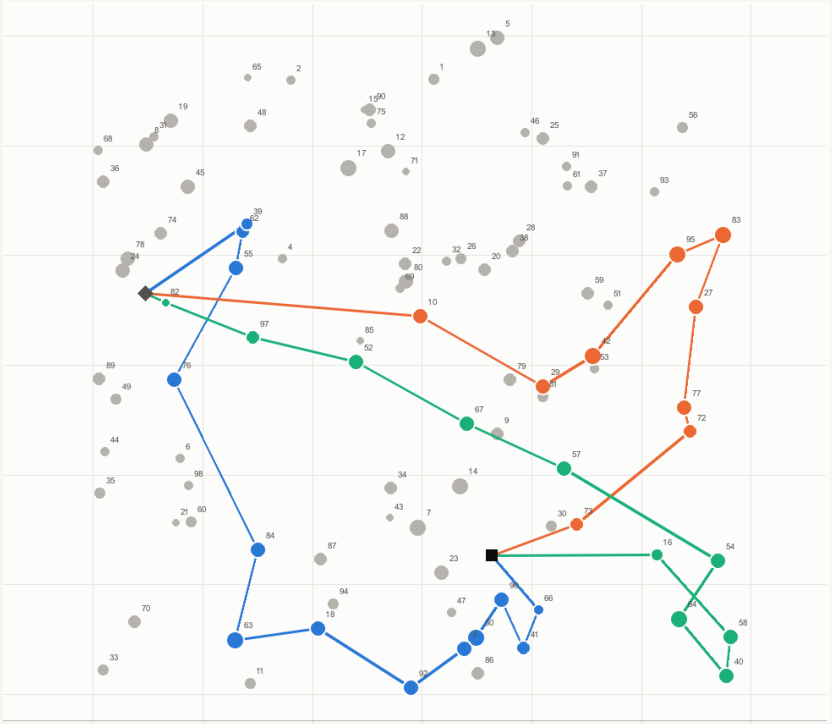
instances where the winner was not the best solution before local search

Grid: 21 α × (1 deterministic + 10 geometric + 10 softmax runs of 10 iterations) per instance, then elite and local search. 140 s for all 387 instances on 12 threads.

One instance, before and after: p4.3.k

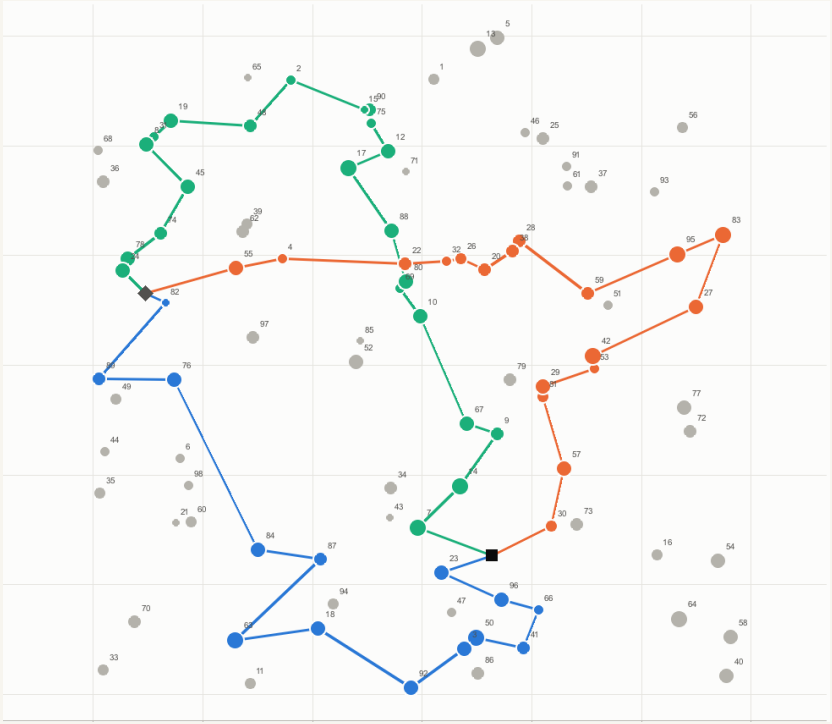
PJS, best α = 0.40

reward 614 · 32 of 98 customers



GRASP + local search

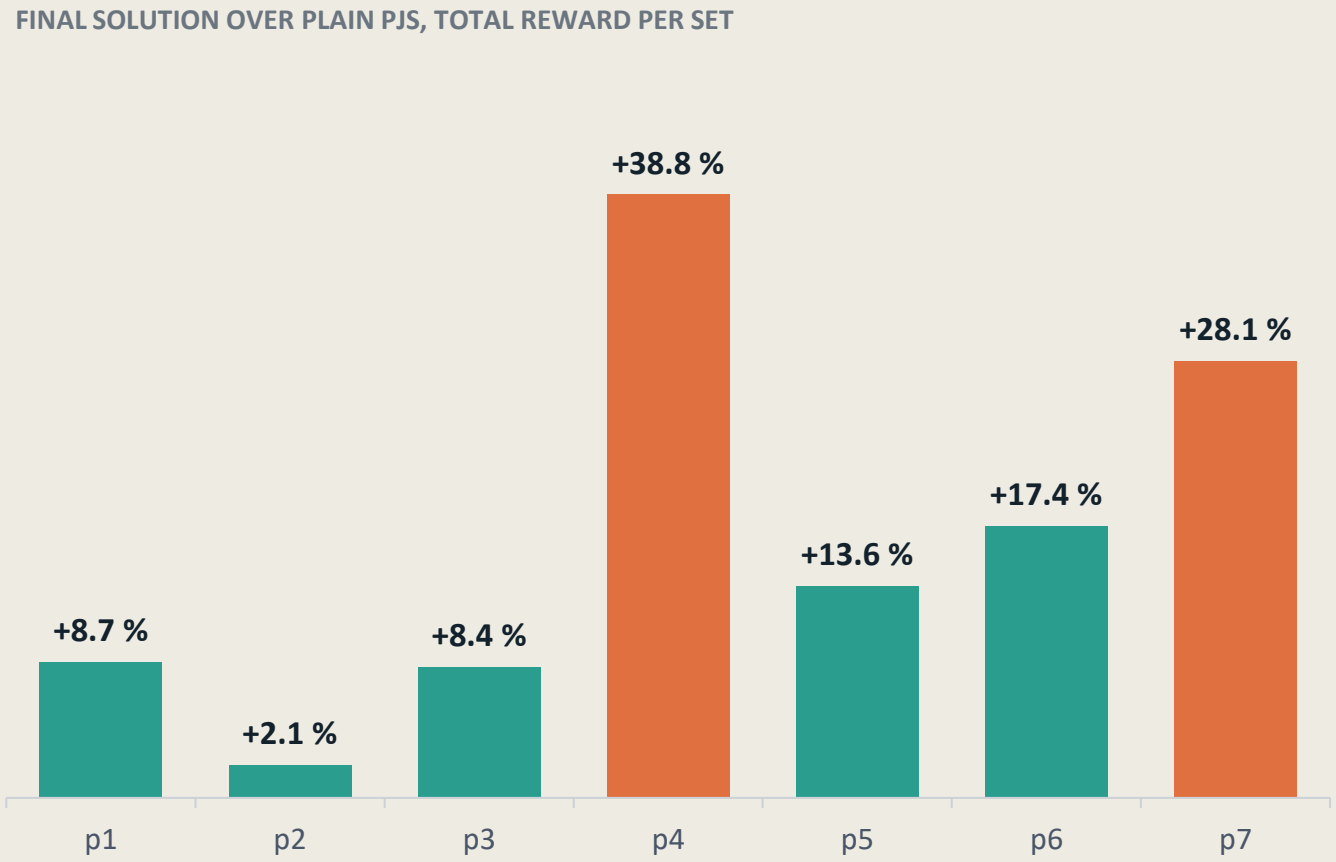
reward 874 · 54 of 98 customers



+42 %: same three vehicles, same time limit, 22 more customers.

m = 3, tmax = 50. Final solution from a geometric run with α = 0.45: reward 625 before local search, 19 customers inserted. Marker area ∝ reward.

The gain grows with the instance



Small sets: little room

p1–p3 have 21 to 33 nodes, so few customers are left to insert.

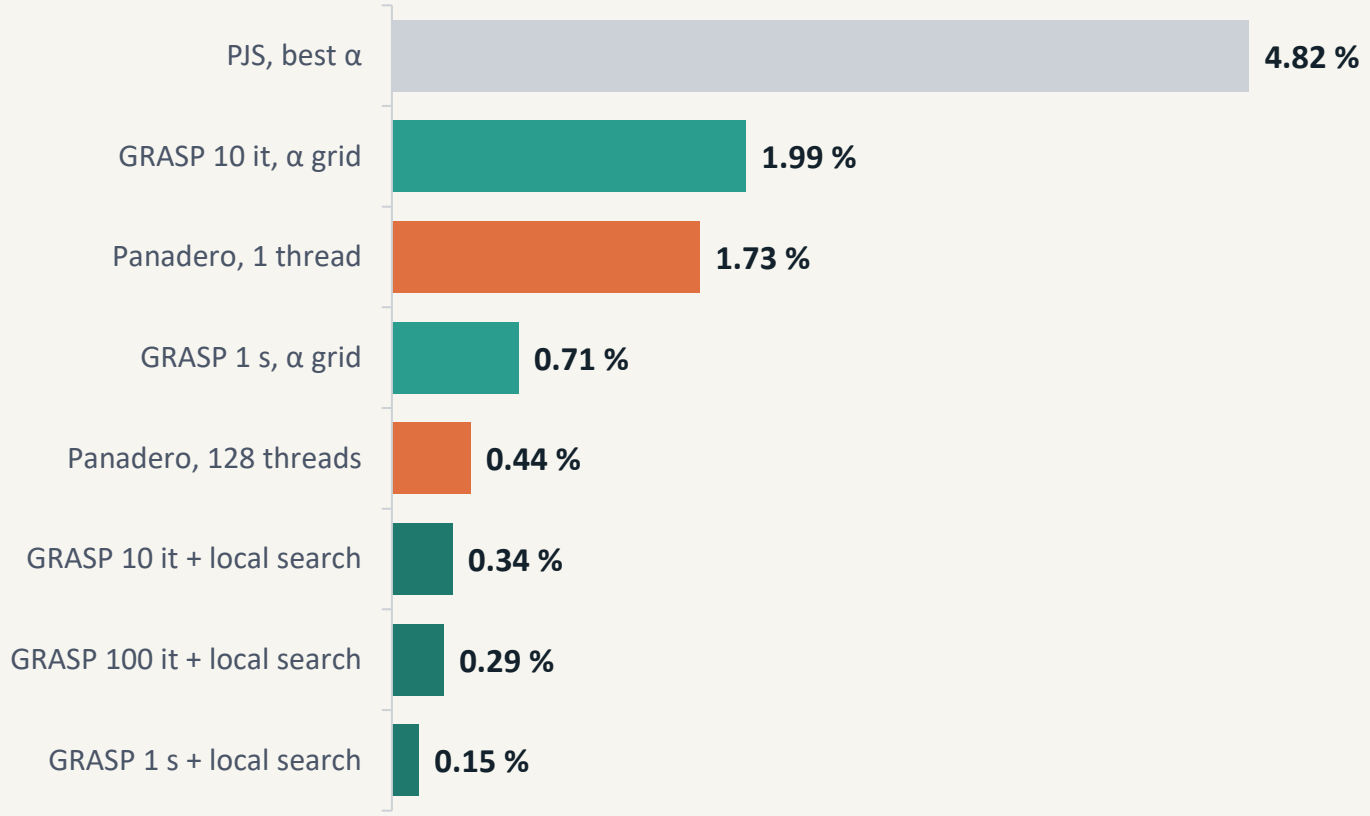
p4 and p7: about 100 nodes

Long routes, many customers left out. Local search inserts up to 49 customers in one solution.

Panadero et al. tabulate instance by instance only p1 and p2, where the margin is smallest; for p3–p7 they give set means.

p1 and p2, instance by instance: ahead of Panadero

MEAN GAP TO THE BEST KNOWN SOLUTION, 81 INSTANCES OF p1–p2, LOWER IS BETTER



BEST KNOWN SOLUTION REACHED

69 / 81

Panadero, 128 threads

73 / 81

Ours, 10 it + local search

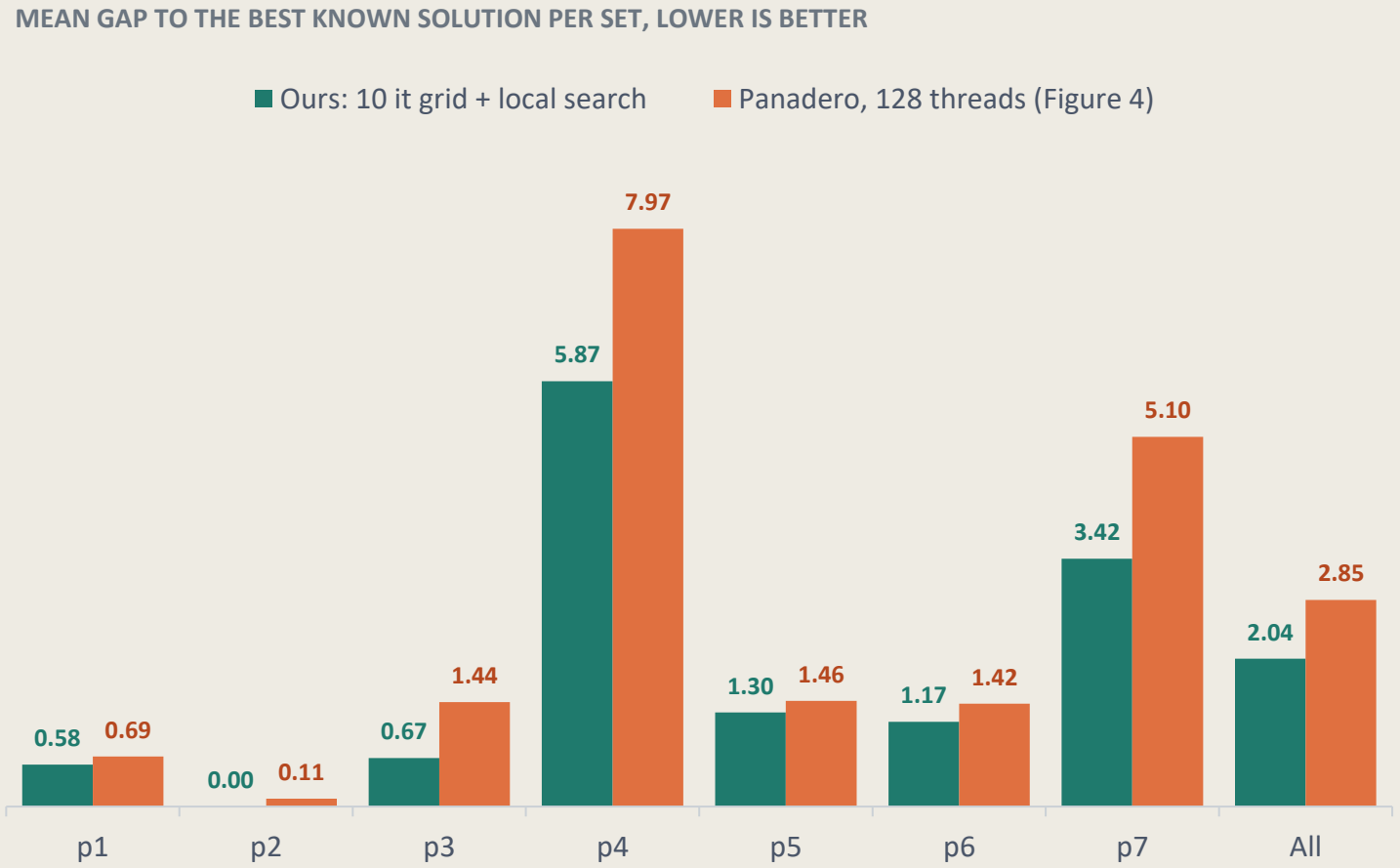
77 / 81

Ours, 1 s + local search

None of our solutions beats a best known value.

Gap = 100 · (BKS − ours) / BKS, averaged over instances, as in the paper's Tables 3 and 4 (p2.4.a is printed there as “p2.4.1”).

All seven sets: below Panadero's gap in every one



2.04 %

our mean gap over the 353 instances, against 2.85 % for Panadero with 128 threads

0 / 353

final solutions above the best known value: no sign of an infeasible route in any set

189 / 353

best known value reached, 33 of 33 in p2, 5 of 56 in p4

BKS from the PSOiA results of Dang, Guibadj and Moukrim (equal to Panadero's on p1–p2). Panadero's “All”: Figure 4 weighted by instances.

Trying many α beats iterating longer with one

1 s with the single best α

≈ 900,000

constructions per instance
20 runs × ~45,000 iterations

Mean gap to BKS, no local search: **1.49 %**

100 iterations on the α grid

42,000

constructions per instance
21 α × 20 runs × 100 iterations

Mean gap to BKS, no local search: **1.45 %**

Same quality with about 20 times less work.

A small change in α reorders the savings list and the greedy merge builds different routes. Spreading the budget over α explores more than spending it on one.

One 1 s run of ours alone averages 2.3–2.4 % against 1.73 % for one thread of Panadero.

TAKEAWAY

Diversity first, then local search.

21 α and two randomisations give the elite; 2-opt and insertion turn them into the final routes.

WHAT WE FOUND

+20 % reward over plain PJS on all 353 feasible instances.

Below Panadero with 128 threads in all 7 sets: 2.04 % from the best known values, against 2.85 %.

The paper's rule for α does not hold on these sets.

WORTH ASKING ABOUT

Not equal compute: the 1 s grid spends ~420 CPU-seconds per instance, more than Panadero's 128.

1 s runs depend on the machine's load, so they are not exactly reproducible.

Moves between routes and swaps are still open. For p3–p7 the paper only publishes set means.

Thank you — questions?

BACKUP

All variants against Panadero et al.

Method	Mean gap	BKS reached	vs Panadero 128 (better / equal / worse)
PJS, best α	4.82 %	44	0 / 45 / 36
GRASP 10 it, α grid	1.99 %	53	1 / 56 / 24
Panadero, 1 thread	1.73 %	52	—
GRASP 1 s, best α	1.49 %	56	2 / 60 / 19
GRASP 100 it, α grid	1.45 %	57	3 / 57 / 21
GRASP 1 s, α grid	0.71 %	65	4 / 67 / 10
GRASP 1 s, best α + local search	0.71 %	63	7 / 62 / 12
Panadero, 128 threads	0.44 %	69	—
GRASP 10 it, α grid + local search	0.34 %	73	10 / 65 / 6
GRASP 100 it, α grid + local search	0.29 %	74	12 / 62 / 7
GRASP 1 s, α grid + local search	**0.15 %**	**77**	**12 / 65 / 4**

Sources: docs/comparacion_panadero*.md and results/best_solution.csv. Panadero rows from Tables 3 and 4 of the paper.